

PROGRAM 5

TRANSFER LEARNING USING RESNET50 AND CNN ARCHITECTURE COMPARISON

OBJECTIVE

To perform Image Classification using a pre-trained ResNet50 model and compare its performance with a Custom CNN Architecture.

In this program students will learn:

- ✓ Reading Dataset from Folders
- ✓ Loading ResNet50 Architecture
- ✓ Loading .weights.h5 Files
- ✓ Transfer Learning
- ✓ Custom CNN Design
- ✓ Architecture Comparison
- ✓ Feature Extraction
- ✓ Model Evaluation
- ✓ Accuracy and Loss Analysis

MODEL FILE REQUIRED

Models/

└── resnet50.weights.h5

DATASET FOLDER STRUCTURE

Vehicles/

```
|— Car/
|   |— image1.jpg
|   |— image2.jpg
|   └─ ...
|
|— Bike/
|   |— image1.jpg
|   |— image2.jpg
|   └─ ...
|
└─ Bus/
```

```
|— image1.jpg
|— image2.jpg
└─ ...
```

PART A LOAD DATASET

STEP 1

IMPORT LIBRARIES

Code:

```
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt

from tensorflow.keras.preprocessing.image import ImageDataGenerator

from tensorflow.keras.models import Sequential
from tensorflow.keras.models import Model

from tensorflow.keras.layers import Conv2D
from tensorflow.keras.layers import MaxPooling2D
from tensorflow.keras.layers import Flatten
from tensorflow.keras.layers import Dense
from tensorflow.keras.layers import Dropout
from tensorflow.keras.layers import GlobalAveragePooling2D

from tensorflow.keras.applications import ResNet50

print("Libraries Imported")
```

STEP 2

LOAD DATASET

Code:

```
datagen = ImageDataGenerator(
    rescale=1./255,
    validation_split=0.2
)

train_data = datagen.flow_from_directory(
    "Vehicles",
    target_size=(224,224),
    batch_size=32,
    class_mode='categorical',
    subset='training'
)

val_data = datagen.flow_from_directory(
    "Vehicles",
    target_size=(224,224),
    batch_size=32,
    class_mode='categorical',
```

```
        subset='validation'  
    )  
  
    print("Dataset Loaded")
```

Explanation:

Images are resized to:

224 × 224

because ResNet50 requires this input size.

STEP 3

DISPLAY SAMPLE IMAGES

Code:

```
images, labels = next(train_data)  
  
plt.figure(figsize=(10,5))  
  
for i in range(6):  
  
    plt.subplot(2,3,i+1)  
  
    plt.imshow(images[i])  
  
    plt.axis("off")  
  
plt.show()
```

PART B

CUSTOM CNN MODEL

STEP 4

CREATE CNN MODEL

Code:

```
cnn_model = Sequential([

    Conv2D(
        32,
        (3,3),
        activation='relu',
        input_shape=(224,224,3)
    ),

    MaxPooling2D(),

    Conv2D(
        64,
        (3,3),
        activation='relu'
    ),

    MaxPooling2D(),

    Flatten(),

    Dense(
        128,
        activation='relu'
    ),

    Dense(
        3,
        activation='softmax'
    )

])
```

STEP 5

COMPILE CNN

Code:

```
cnn_model.compile(  
optimizer='adam',  
loss='categorical_crossentropy',  
metrics=['accuracy']  
)
```

STEP 6

TRAIN CNN

Code:

```
cnn_history = cnn_model.fit(  
train_data,  
validation_data=val_data,  
epochs=5  
)
```

STEP 7

EVALUATE CNN

Code:

```
cnn_loss,cnn_acc = cnn_model.evaluate(  
val_data  
)  
  
print(  
"CNN Accuracy =",  
cnn_acc  
)
```

PART C

TRANSFER LEARNING USING RESNET50

STEP 8

LOAD RESNET50

Code:

```
base_model = ResNet50(  
    weights=None,  
    include_top=False,  
    input_shape=(224,224,3)  
)  
  
print("ResNet50 Loaded")
```

Explanation:

weights=None

Prevents internet download.

include_top=False

Removes original classifier.

STEP 9

LOAD .WEIGHTS.H5 FILE

Code:

```
base_model.load_weights(  
    "Models/resnet50.weights.h5"  
)  
  
print("Weights Loaded")
```

STEP 10

FREEZE PRETRAINED LAYERS

Code:

```
for layer in base_model.layers:  
    layer.trainable=False
```

Explanation:

Pretrained weights remain unchanged.

STEP 11

ADD CUSTOM CLASSIFIER

Code:

```
x = GlobalAveragePooling2D()(  
    base_model.output  
)  
  
x = Dense(  
    128,  
    activation='relu'  
) (x)  
  
x = Dropout(0.3)(x)  
  
output = Dense(  
    3,  
    activation='softmax'  
) (x)  
  
resnet_model = Model(  
    inputs=base_model.input,  
    outputs=output  
)
```

STEP 12

VIEW MODEL SUMMARY

Code:

```
resnet_model.summary()
```

STEP 13

COMPILE RESNET50 MODEL

Code:

```
resnet_model.compile(  
optimizer='adam',  
loss='categorical_crossentropy',  
metrics=['accuracy']  
)
```

STEP 14

TRAIN RESNET50 MODEL

Code:

```
resnet_history = resnet_model.fit(  
train_data,  
validation_data=val_data,  
epochs=5  
)
```

STEP 15

EVALUATE RESNET50 MODEL

Code:

```
resnet_loss, resnet_acc = resnet_model.evaluate(  
val_data  
)  
  
print(
```

```
"ResNet50 Accuracy =",  
resnet_acc  
)
```

PART D

COMPARE BOTH MODELS

STEP 16

COMPARE ACCURACY

Code:

```
print(  
    "Custom CNN Accuracy =",  
    cnn_acc  
)  
  
print(  
    "ResNet50 Accuracy =",  
    resnet_acc  
)
```

STEP 17

PLOT CNN ACCURACY

Code:

```
plt.plot(  
    cnn_history.history['accuracy']  
)  
  
plt.plot(  
    cnn_history.history['val_accuracy']  
)  
  
plt.title("CNN Accuracy")  
  
plt.legend([
```

```
'Training',  
'Validation'  
)  
  
plt.show()
```

STEP 18

PLOT RESNET50 ACCURACY

Code:

```
plt.plot(  
    resnet_history.history['accuracy']  
)  
  
plt.plot(  
    resnet_history.history['val_accuracy']  
)  
  
plt.title("ResNet50 Accuracy")  
  
plt.legend([  
    'Training',  
    'Validation'  
)  
  
plt.show()
```

STEP 19

PREDICT NEW IMAGE

Code:

```
from tensorflow.keras.preprocessing import image  
  
img = image.load_img(  
    "test.jpg",  
    target_size=(224,224)  
)
```

```
img_array = image.img_to_array(img)

img_array = img_array/255.0

img_array = np.expand_dims(
    img_array,
    axis=0
)

prediction = resnet_model.predict(
    img_array
)

print(prediction)
```

UNDERSTANDING RESNET50

Traditional CNN

↓

Learns From Scratch

↓

Needs More Data

ResNet50

↓

Already Learned Millions of Features

↓

Uses Residual Connections

↓

Faster Convergence



Higher Accuracy

KEY CONCEPTS LEARNED

- ✓ ResNet50
 - ✓ Transfer Learning
 - ✓ Feature Extraction
 - ✓ Residual Network
 - ✓ Loading .weights.h5
 - ✓ Freezing Layers
 - ✓ Architecture Comparison
 - ✓ CNN Design
 - ✓ Accuracy Analysis
 - ✓ Performance Comparison
 - ✓ Prediction
-

EXPECTED VIVA QUESTIONS

1. What is ResNet50?
2. What are Residual Connections?
3. Why use Transfer Learning?
4. Why freeze layers?
5. Difference between CNN and ResNet50?
6. Why is GlobalAveragePooling used?

7. What is Feature Extraction?
 8. Why does ResNet50 perform better on complex datasets?
 9. What factors influence architecture selection?
 10. When should Transfer Learning be preferred over training from scratch?
-

FINAL STUDY PLAN

Program 1 → Learn Folder Dataset Reading, CNN, Training and Evaluation.

Program 2 → Learn YOLO, Object Detection, Bounding Boxes and Confidence Scores.

Program 3 → Learn U-Net, Masks, Encoder, Decoder and Segmentation.

Program 4 → Learn VGG16, Transfer Learning and Loading .weights.h5.

Program 5 → Learn ResNet50, Architecture Selection and Model Comparison.

After practicing these five programs, students should be able to understand most coding patterns used in Deep Learning laboratory exercises.